



Needleman–Wunsch algorithm

The **Needleman–Wunsch algorithm** is an algorithm used in bioinformatics to align protein or nucleotide sequences. It was one of the first applications of dynamic programming to compare biological sequences. The algorithm was developed by Saul B. Needleman and Christian D. Wunsch and published in 1970.^[1] The algorithm essentially divides a large problem (e.g. the full sequence) into a series of smaller problems, and it uses the solutions to the smaller problems to find an optimal solution to the larger problem.^[2] It is also sometimes referred to as the optimal matching algorithm and the global alignment technique. The Needleman–Wunsch algorithm is still widely used for optimal global alignment, particularly when the quality of the global alignment is of the utmost importance. The algorithm assigns a score to every possible alignment, and the purpose of the algorithm is to find all possible alignments having the highest score.

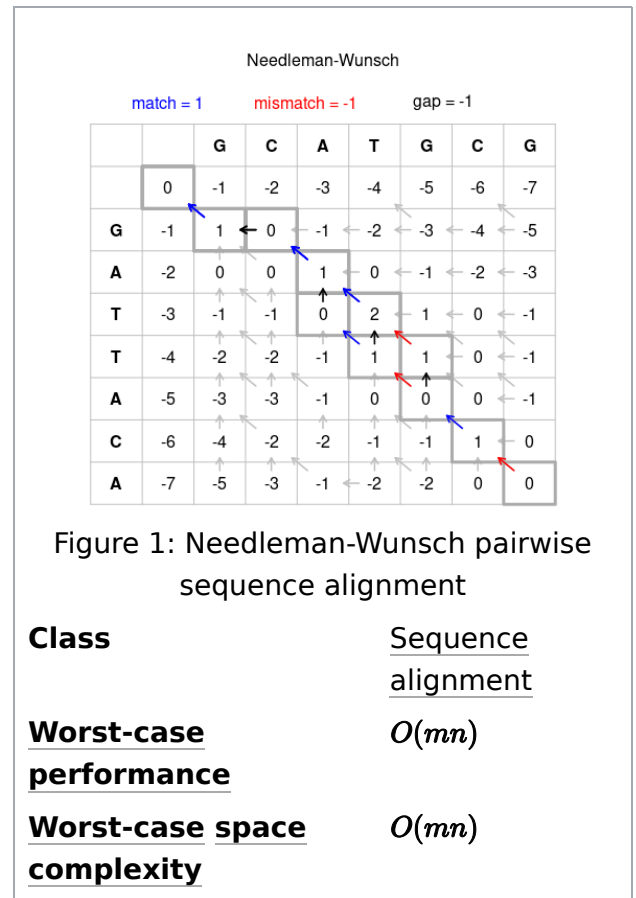
Introduction

This algorithm can be used for any two strings. This guide will use two small DNA sequences as examples as shown in Figure 1:

GCATGCG
GATTACA

Constructing the grid

First construct a grid such as one shown in Figure 1 above. Start the first string in the top of the third column and start the other string at the start of the third row. Fill out the rest of the column and row headers as in Figure 1. There should be no numbers in the grid yet.



		G	C	A	T	G	C	G
G								
A								
T								
T								
A								
C								
A								

Choosing a scoring system

Next, decide how to score each individual pair of letters. Using the example above, one possible alignment candidate might be:

```
12345678
GCATG-CG
G-ATTACA
```

The letters may match, mismatch, or be matched to a gap (a deletion or insertion (indel)):

- **Match:** The two letters at the current index are the same.
- **Mismatch:** The two letters at the current index are different.
- **Indel (Insertion or Deletion):** The best alignment involves one letter aligning to a gap in the other string.

Each of these scenarios is assigned a score and the sum of the scores of all the pairings is the score of the whole alignment candidate. Different systems exist for assigning scores; some have been outlined in the [Scoring systems](#) section below. For now, the system used by Needleman and Wunsch^[1] will be used:

- **Match:** +1
- **Mismatch or Indel:** -1

For the Example above, the score of the alignment would be 0:

```
GCATG-CG
G-ATTACA
+---+---+ -> 1*4 + (-1)*4 = 0
```

Filling in the table

Start with a zero in the first row, first column (not including the cells containing nucleotides). Move through the cells row by row, calculating the score for each cell. The score is calculated by comparing the scores of the cells neighboring to the left, top or top-left (diagonal) of the cell and adding the appropriate score for match, mismatch or indel. Take the maximum of the candidate scores for each of the three possibilities:

- The path from the top or left cell represents an indel pairing, so take the scores

of the left and the top cell, and add the score for indel to each of them.

- The diagonal path represents a match/mismatch, so take the score of the top-left diagonal cell and add the score for match if the corresponding bases (letters) in the row and column are matching or the score for mismatch if they do not.

The resulting score for the cell is the highest of the three candidate scores.

Given there is no 'top' or 'top-left' cells for the first row only the existing cell to the left can be used to calculate the score of each cell. Hence -1 is added for each shift to the right as this represents an indel from the previous score. This results in the first row being 0, -1 , -2 , -3 , -4 , -5 , -6 , -7 . The same applies to the first column as only the existing score above each cell can be used. Thus the resulting table is:

		G	C	A	T	G	C	G
	0	-1	-2	-3	-4	-5	-6	-7
G	-1							
A	-2							
T	-3							
T	-4							
A	-5							
C	-6							
A	-7							

The first case with existing scores in all 3 directions is the intersection of our first letters (in this case G and G). The surrounding cells are below:

		G
	0	-1
G	-1	X

This cell has three possible candidate sums:

- The diagonal top-left neighbor has score 0. The pairing of G and G is a match, so add the score for match: $0+1 = 1$
- The top neighbor has score -1 and moving from there represents an indel, so add the score for indel: $(-1) + (-1) = (-2)$
- The left neighbor also has score -1 , represents an indel and also produces (-2) .

The highest candidate is 1 and is entered into the cell:

		G
	0	-1
G	-1	1

The cell which gave the highest candidate score must also be recorded. In the completed diagram in figure 1 above, this is represented as an arrow from the cell in row and column 2 to the cell in row and column 1.

In the next example, the diagonal step for both X and Y represents a mismatch:

		G	C
	0	-1	-2
G	-1	1	X
A	-2	Y	

X:

- Top: $(-2) + (-1) = (-3)$
- Left: $(+1) + (-1) = (0)$
- Top-Left: $(-1) + (-1) = (-2)$

Y:

- Top: $(1) + (-1) = (0)$
- Left: $(-2) + (-1) = (-3)$
- Top-Left: $(-1) + (-1) = (-2)$

For both X and Y, the highest score is zero:

		G	C
	0	-1	-2
G	-1	1	0
A	-2	0	

The highest candidate score may be reached by two of the neighboring cells:

	T	G
T	1	1
A	0	X

- Top: $(1) + (-1) = (0)$
- Top-Left: $(1) + (-1) = (0)$
- Left: $(0) + (-1) = (-1)$

In this case, all directions reaching the highest candidate score must be noted as possible origin cells in the finished diagram in figure 1, e.g. in the cell in row and column 6.

Filling in the table in this manner gives the scores of all possible alignment candidates, the score in the cell on the bottom right represents the alignment score for the best alignment.

Tracing arrows back to origin

Mark a path from the cell on the bottom right back to the cell on the top left by following the direction of the arrows. From this path, the sequence is constructed by these rules:

- A diagonal arrow represents a match or mismatch, so the letter of the column and the letter of the row of the origin cell will align.
- A horizontal or vertical arrow represents an indel. Vertical arrows will align a gap

("-") to the letter of the row (the "side" sequence), horizontal arrows will align a gap to the letter of the column (the "top" sequence).

- If there are multiple arrows to choose from, they represent a branching of the alignments. If two or more branches all belong to paths from the bottom right to the top left cell, they are equally viable alignments. In this case, note the paths as separate alignment candidates.

Following these rules, the steps for one possible alignment candidate in figure 1 are:

```
G → CG → GCG → -GCG → T-GCG → AT-GCG → CAT-GCG → GCAT-GCG
A → CA → ACA → TACA → TTACA → ATTACA → -ATTACA → G-ATTACA
      ↓
(branch) → TGCG → -TGCG → ...
          → TACA → TTACA → ...
```

Scoring systems

Basic scoring schemes

The simplest scoring schemes simply give a value for each match, mismatch and indel. The step-by-step guide above uses match = 1, mismatch = -1, indel = -1. Thus the lower the alignment score the larger the edit distance, for this scoring system one wants a high score. Another scoring system might be:

- Match = 0
- Indel = -1
- Mismatch = -1

For this system the alignment score will represent the edit distance between the two strings. Different scoring systems can be devised for different situations, for example if gaps are considered very bad for your alignment you may use a scoring system that penalises gaps heavily, such as:

- Match = 1
- Indel = -10
- Mismatch = -1

Similarity matrix

More complicated scoring systems attribute values not only for the type of alteration, but also for the letters that are involved. For example, a match between A and A may be given 1, but a match between T and T may be given 4. Here (assuming the first scoring system) more importance is given to the Ts matching than the As, i.e. the Ts matching is assumed to be more significant to the alignment. This weighting based on letters also applies to mismatches.

In order to represent all the possible combinations of letters and their resulting scores a similarity matrix is used. The similarity matrix for the most basic system is represented as:

	A	G	C	T
A	1	-1	-1	-1
G	-1	1	-1	-1
C	-1	-1	1	-1
T	-1	-1	-1	1

Each score represents a switch from one of the letters the cell matches to the other. Hence this represents all possible matches and mismatches (for an alphabet of ACGT). Note all the matches go along the diagonal, also not all the table needs to be filled, only this triangle because the scores are reciprocal. = (Score for $A \rightarrow C$ = Score for $C \rightarrow A$). If implementing the T-T = 4 rule from above the following similarity matrix is produced:

	A	G	C	T
A	1	-1	-1	-1
G	-1	1	-1	-1
C	-1	-1	1	-1
T	-1	-1	-1	4

Different scoring matrices have been statistically constructed which give weight to different actions appropriate to a particular scenario. Having weighted scoring matrices is particularly important in protein sequence alignment due to the varying frequency of the different amino acids. There are two broad families of scoring matrices, each with further alterations for specific scenarios:

- PAM
- BLOSUM

Gap penalty

When aligning sequences there are often gaps (i.e. indels), sometimes large ones. Biologically, a large gap is more likely to occur as one large deletion as opposed to multiple single deletions. Hence two small indels should have a worse score than one large one. The simple and common way to do this is via a large gap-start score for a new indel and a smaller gap-extension score for every letter which extends the indel. For example, new-indel may cost -5 and extend-indel may cost -1. In this way an alignment such as:

```
GAAAAAAT
G--A-A-T
```

which has multiple equal alignments, some with multiple small alignments will now align as:

```
GAAAAAAT
GAA----T
```

or any alignment with a 4 long gap in preference over multiple small gaps.

Advanced presentation of algorithm

Scores for aligned characters are specified by a similarity matrix. Here, $S(a, b)$ is the similarity of characters a and b . It uses a linear gap penalty, here called d .

For example, if the similarity matrix was

	A	G	C	T
A	10	-1	-3	-4
G	-1	7	-5	-3
C	-3	-5	9	0
T	-4	-3	0	8

then the alignment:

```
AGACTAGTTAC
CGA---GACGT
```

with a gap penalty of -5 , would have the following score:

$$S(A,C) + S(G,G) + S(A,A) + (3 \times d) + S(G,G) + S(T,A) + S(T,C) + S(A,G) + S(C,T) \\ = -3 + 7 + 10 - (3 \times 5) + 7 + (-4) + 0 + (-1) + 0 = 1$$

To find the alignment with the highest score, a two-dimensional array (or matrix) F is allocated. The entry in row i and column j is denoted here by F_{ij} . There is one row for each character in sequence A , and one column for each character in sequence B . Thus, if aligning sequences of sizes n and m , the amount of memory used is in $O(nm)$. Hirschberg's algorithm only holds a subset of the array in memory and uses $\Theta(\min\{n, m\})$ space, but is otherwise similar to Needleman-Wunsch (and still requires $O(nm)$ time).

As the algorithm progresses, the F_{ij} will be assigned to be the optimal score for the alignment of the first $i = 0, \dots, n$ characters in A and the first $j = 0, \dots, m$ characters in B . The principle of optimality is then applied as follows:

- Basis:

$$F_{0j} = d * j$$

$$F_{i0} = d * i$$

- Recursion, based on the principle of optimality:

$$F_{ij} = \max(F_{i-1,j-1} + S(A_i, B_j), F_{i,j-1} + d, F_{i-1,j} + d)$$

The pseudo-code for the algorithm to compute the F matrix therefore looks like this:

```
d ← Gap penalty score
for i = 0 to length(A)
  F(i,0) ← d * i
for j = 0 to length(B)
  F(0,j) ← d * j
for i = 1 to length(A)
  for j = 1 to length(B)
  {
    Match ← F(i-1, j-1) + S(Ai, Bj)
    Delete ← F(i-1, j) + d
    Insert ← F(i, j-1) + d
```

```

    F(i,j) ← max(Match, Insert, Delete)
}

```

Once the F matrix is computed, the entry F_{nm} gives the maximum score among all possible alignments. To compute an alignment that actually gives this score, you start from the bottom right cell, and compare the value with the three possible sources (Match, Insert, and Delete above) to see which it came from. If Match, then A_i and B_j are aligned, if Delete, then A_i is aligned with a gap, and if Insert, then B_j is aligned with a gap. (In general, more than one choice may have the same value, leading to alternative optimal alignments.)

```

AlignmentA ← ""
AlignmentB ← ""
i ← length(A)
j ← length(B)
while (i > 0 or j > 0)
{
    if (i > 0 and j > 0 and F(i, j) == F(i-1, j-1) + S(Ai, Bj))
    {
        AlignmentA ← Ai + AlignmentA
        AlignmentB ← Bj + AlignmentB
        i ← i - 1
        j ← j - 1
    }
    else if (i > 0 and F(i, j) == F(i-1, j) + d)
    {
        AlignmentA ← Ai + AlignmentA
        AlignmentB ← "-" + AlignmentB
        i ← i - 1
    }
    else
    {
        AlignmentA ← "-" + AlignmentA
        AlignmentB ← Bj + AlignmentB
        j ← j - 1
    }
}

```

Complexity

Computing the score F_{ij} for each cell in the table is an $O(1)$ operation. Thus the time complexity of the algorithm for two sequences of length n and m is $O(mn)$.^[3] It has been shown that it is possible to improve the running time to $O(mn/\log n)$ using the Method of Four Russians.^{[3][4]} Since the algorithm fills an $n \times m$ table the space complexity is $O(mn)$.^[3]

Historical notes and algorithm development

The original purpose of the algorithm described by Needleman and Wunsch was to find similarities in the amino acid sequences of two proteins.^[1]

Needleman and Wunsch describe their algorithm explicitly for the case when the alignment is penalized solely by the matches and mismatches, and gaps have no penalty ($d=0$). The original publication from 1970 suggests the recursion $F_{ij} = \max_{h < i, k < j} \{F_{h,j-1} + S(A_i, B_j), F_{i-1,k} + S(A_i, B_j)\}$.

The corresponding dynamic programming algorithm takes cubic time. The paper also points out that the recursion can accommodate arbitrary gap penalization formulas:

A penalty factor, a number subtracted for every gap made, may be assessed as a barrier to allowing the gap. The penalty factor could be a function of the size and/or direction of the gap. [page 444]

A better dynamic programming algorithm with quadratic running time for the same problem (no gap penalty) was introduced later^[5] by David Sankoff in 1972. Similar quadratic-time algorithms were discovered independently by T. K. Vintsyuk^[6] in 1968 for speech processing ("time warping"), and by Robert A. Wagner and Michael J. Fischer^[7] in 1974 for string matching.

Needleman and Wunsch formulated their problem in terms of maximizing similarity. Another possibility is to minimize the edit distance between sequences, introduced by Vladimir Levenshtein. Peter H. Sellers showed^[8] in 1974 that the two problems are equivalent.

The Needleman–Wunsch algorithm is still widely used for optimal global alignment, particularly when the quality of the global alignment is of the utmost importance. However, the algorithm is expensive with respect to time and space, proportional to the product of the length of two sequences and hence is not suitable for long sequences.

Recent development has focused on improving the time and space cost of the algorithm while maintaining quality. For example, in 2013, a Fast Optimal Global Sequence Alignment Algorithm (FOGSAA),^[9] suggested alignment of nucleotide/protein sequences faster than other optimal global alignment methods, including the Needleman–Wunsch algorithm. The paper claims that when compared to the Needleman–Wunsch algorithm, FOGSAA achieves a time gain of 70–90% for highly similar nucleotide sequences (with > 80% similarity), and 54–70% for sequences having 30–80% similarity.

Applications outside bioinformatics

Computer stereo vision

Stereo matching is an essential step in the process of 3D reconstruction from a pair of stereo images. When images have been rectified, an analogy can be drawn between aligning nucleotide and protein sequences and matching pixels belonging to scan lines, since both tasks aim at establishing optimal correspondence between two strings of characters.

Although in many applications image rectification can be performed, e.g. by camera resectioning or calibration, it is sometimes impossible or impractical since the computational cost of accurate rectification models prohibit their usage in real-time applications. Moreover, none of these models is suitable when a camera lens displays unexpected distortions, such as those generated by raindrops, weatherproof covers or dust. By extending the Needleman–Wunsch algorithm, a line in the 'left' image can be associated to a curve in the 'right' image by finding the alignment with the highest score in a three-dimensional array (or matrix). Experiments demonstrated that such extension allows dense pixel matching between unrectified or distorted images.^[10]

Artificial neural networks

The Needleman–Wunsch algorithm has also been applied to the comparison of artificial neural network architectures. Inspired by how global sequence alignment identifies similarities in the aforementioned

biological sequences, this approach represents neural networks as sequences of computational layers or blocks. By aligning these sequences, architectural similarity can be quantified in a principled manner.

This method has been successfully used in neural architecture search (NAS), particularly in approaches based on evolutionary algorithms. Architectural similarity can be assessed without requiring model training, enabling more efficient guidance of diversity during population-based search, preventing premature convergence.^[11]

See also

- Wagner–Fischer algorithm
- Smith–Waterman algorithm
- Sequence mining
- Levenshtein distance
- Dynamic time warping
- Sequence alignment

References

1. Needleman, Saul B. & Wunsch, Christian D. (1970). "A general method applicable to the search for similarities in the amino acid sequence of two proteins". *Journal of Molecular Biology*. **48** (3): 443–53. doi:10.1016/0022-2836(70)90057-4 (<https://doi.org/10.1016%2F0022-2836%2870%2990057-4>). PMID 5420325 (<https://pubmed.ncbi.nlm.nih.gov/5420325>).
2. "bioinformatics" (<http://www.britannica.com/EBchecked/topic/1334661/bioinformatics/285871/Goals-of-bioinformatics#ref1115380>). Retrieved 10 September 2014.
3. Wing-Kin., Sung (2010). *Algorithms in bioinformatics : a practical introduction*. Boca Raton: Chapman & Hall/CRC Press. pp. 34–35. ISBN 9781420070330. OCLC 429634761 (<https://search.worldcat.org/oclc/429634761>).
4. Masek, William; Paterson, Michael (February 1980). "A faster algorithm computing string edit distances" (<https://doi.org/10.1016%2F0022-0000%2880%2990002-1>). *Journal of Computer and System Sciences*. **20**: 18–31. doi:10.1016/0022-0000(80)90002-1 (<https://doi.org/10.1016%2F0022-0000%2880%2990002-1>). hdl:1721.1/148933 (<https://hdl.handle.net/1721.1%2F148933>).
5. Sankoff D (1972). "Matching sequences under deletion/insertion constraints" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC427531>). *Proceedings of the National Academy of Sciences of the USA*. **69** (1): 4–6. Bibcode:1972PNAS...69....4S (<http://ui.adsabs.harvard.edu/abs/1972PNAS...69....4S>). doi:10.1073/pnas.69.1.4 (<https://doi.org/10.1073%2Fpnas.69.1.4>). PMC 427531 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC427531>). PMID 4500555 (<https://pubmed.ncbi.nlm.nih.gov/4500555>).
6. Vintsyuk TK (1968). "Speech discrimination by dynamic programming". *Kibernetika*. **4**: 81–88. doi:10.1007/BF01074755 (<https://doi.org/10.1007%2FBF01074755>). S2CID 123081024 (<https://api.semanticscholar.org/CorpusID:123081024>).

7. Wagner RA, Fischer MJ (1974). "The string-to-string correction problem" (<https://doi.org/10.1145%2F321796.321811>). *Journal of the ACM*. **21** (1): 168–173. doi:10.1145/321796.321811 (<https://doi.org/10.1145%2F321796.321811>). S2CID 13381535 (<https://api.semanticscholar.org/CorpusID:13381535>).
8. Sellers PH (1974). "On the theory and computation of evolutionary distances". *SIAM Journal on Applied Mathematics*. **26** (4): 787–793. doi:10.1137/0126070 (<https://doi.org/10.1137%2F0126070>).
9. Chakraborty, Angana; Bandyopadhyay, Sanghamitra (29 April 2013). "FOGSAA: Fast Optimal Global Sequence Alignment Algorithm" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3638164>). *Scientific Reports*. **3**: 1746. Bibcode:2013NatSR...3.1746C (<https://ui.adsabs.harvard.edu/abs/2013NatSR...3.1746C>). doi:10.1038/srep01746 (<https://doi.org/10.1038%2Fsrep01746>). PMC 3638164 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3638164>). PMID 23624407 (<https://pubmed.ncbi.nlm.nih.gov/23624407>).
10. Thevenon, J; Martinez-del-Rincon, J; Dieny, R; Nebel, J-C (2012). *Dense pixel matching between unrectified and distorted images using dynamic programming* (<https://www.researchgate.net/publication/257928290>). International Conference on Computer Vision Theory and Applications. Rome.
11. Ávila Pava, M; Groh, R; Kist, A M (2024). *Sequence Alignment-based Similarity Metric in Evolutionary Neural Architecture Search* (<https://proceedings.mlr.press/v256/pava24a.html>). International Conference on Automated Machine Learning (AutoML 2024). Paris.

External links

- NW-align: A protein sequence-to-sequence alignment program by Needleman-Wunsch algorithm (online server and source code) (<http://zhanglab.ccmb.med.umich.edu/NW-align>)
- A live demonstration of the algorithm (<https://needleman-wunsch.com>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Needleman-Wunsch_algorithm&oldid=1320393202"